

FUSE User Manual

A Unity Mod Manager modding layer for Railroader

Contents

1. Getting Started
2. Frequently Asked Questions
3. Migrating From Legacy Mods
4. Settings Reference
5. Console Command Reference
6. Troubleshooting
7. Known Issues
8. Installer

Getting Started

This guide takes you from a clean Railroader install to a working FUSE setup with a package loaded.

Migrating from Railroader or another legacy stack? Read `MIGRATION_FROM_LEGACY.md` first — the order of operations matters there.

Before You Start

- **Railroader 2025.1.x.** FUSE targets this line. Other versions are not supported.
- **Unity Mod Manager** for Railroader, installed and working.
- **A backup of your saves.** Any mod that changes the world changes what your saves depend on.

Install FUSE

- 1 Download the latest `FUSE-v<version>.zip` from Releases.
- 2 Extract it so the mod lands at `Railroader/Mods/FUSE`.
- 3 Start Railroader.
- 4 Check that FUSE appears in Unity Mod Manager's mod list and is enabled.

`FUSE-Installer.exe` can do this for you: drop it and your mod zips in the game's base folder and run it. See `FUSE_INSTALLER.md`.

Install Packages

FUSE packages are folders ending in `.FUSE` that go in `Railroader/Mods` alongside FUSE itself.

- 1 Place each `*.FUSE` package folder in `Railroader/Mods`.
- 2 Install any asset packs the package requires — route packages usually depend on at least one.
- 3 Start the game and load a map.

FUSE respects UMM's enabled checkbox for any package folder UMM can see. Unchecking a converted route, audio, or asset package there marks it disabled in FUSE too, and its track and data files are not loaded.

Converting a legacy mod into a `*.FUSE` package is the converter's job — see `FUSE_CONVERTER.md`.

Verify It Worked

Load a map, then open the in-game console and run:

```
/fuse.report
```

A healthy load reports zero faults, conflicts, asset issues, graph issues, and transfer skips. The startup toast shows an abbreviated version of the same thing.

Then confirm your packages are actually present:

```
/fuse.loaded
```

Every package you installed should be listed and applied. A package that is missing here was never discovered — check that it is in `Railroader/Mods` and enabled in UMM. A package listed as faulted was found but failed to

apply; check FUSE.log for the package id and phase.

The FUSE Menu

The FUSE icon in the top bar opens the in-game menu.

- **Status** — the latest load report.
- **Tools** — the Object Inspector, dependency/asset/diagnostics reports, scenery benchmarks, and Runtime Actions (Reload Track/Data, Reload Terrain, Rebuild Caches).

Runtime Actions are recovery and testing tools. Check FUSE.log after using one.

Settings changed through this menu persist to your own settings file and survive mod updates. See SETTINGS.md.

Where The Files Are

What	Where
FUSE log	%USERPROFILE%\AppData\LocalLow\Giraffe Lab LLC\Railroader\FUSE.log
Unity log	%USERPROFILE%\AppData\LocalLow\Giraffe Lab LLC\Railroader\Player.log
Your settings	%USERPROFILE%\AppData\LocalLow\Giraffe Lab LLC\Railroader\FUSE\settings.json
Mods and packages	Railroader\Mods
Dump command output	The main Railroader folder

Paste %USERPROFILE%\AppData\LocalLow\Giraffe Lab LLC\Railroader into Explorer's address bar to jump straight there.

Updating FUSE

- 1 Back up your saves and your current Railroader/Mods/FUSE folder.
- 2 Replace the FUSE mod folder with the new build.
- 3 Re-run the matching converter if the converter or schema version changed.
- 4 Start the game and check /fuse.report.

Keep the converter and the mod in step. Both ship in the same release, so taking them from the same release tag keeps their versions matched.

Uninstalling

- 1 Exit Railroader.
- 2 Remove or disable your *.FUSE packages **first**.
- 3 Remove or disable Railroader/Mods/FUSE.
- 4 Restore your previous mod folders and save backup if needed.
- 5 Start the game and confirm the vanilla game or your legacy stack loads normally.

Order matters: removing FUSE while packages remain leaves package folders nothing will load.

If Something Goes Wrong

Start with TROUBLESHOOTING.md — it maps symptoms to the command that explains them. FAQ.md covers the common questions.

When reporting a problem, include FUSE.log, Player.log, and the output of /fuse.report.

Next Steps

- SETTINGS.md — what you can configure
- CONSOLE_COMMANDS.md — full command reference
- PACKAGE_AUTHOR_GUIDE.md — making your own packages

Frequently Asked Questions

General

What is FUSE?

A Unity Mod Manager modding layer for Railroader. It loads FUSE data packages — route extensions, asset packs, audio packs, track graph changes, world scenery, operations, and progression data — and provides drop-in compatibility for legacy Railroader, Strange Customs, ConfusingSupplements, For Your Convenience, and Alina's Map Mod packages.

Which Railroader version does it support?

The 2025.1.x line. FUSE logs its version report at startup in `FUSE.log`.

Do I need Railroader as well?

No. FUSE replaces it. Running both means two loaders are live at once, and FUSE warns when it finds a leftover `Railroader.dll` or `Railroader.Interchange.dll`.

Is FUSE free and open source?

Yes, under the GNU Affero General Public License v3.0. See `LICENSE`.

Installing And Packages

Where do packages go?

`Railroader/Mods`, alongside FUSE itself. Package folders end in `.FUSE`.

Can I disable a package without deleting it?

Yes — uncheck it in Unity Mod Manager. FUSE respects UMM's enabled checkbox for any package folder UMM can see, and skips its track and data files.

Why isn't my package loading?

Run `/fuse.loaded`. If the package is not listed at all, FUSE never discovered it — check that it is in `Railroader/Mods` and enabled in UMM. If it is listed as faulted, it was found but failed to apply; `FUSE.log` names the package and the phase that failed.

A building or piece of scenery is missing.

Usually a missing asset pack. Run `/fuse.assets` to see which packs FUSE actually found, and install the one the package expects.

Do not work around it by pointing the package at a different asset name. If the correct pack exists, install it.

Does a broken package break everything else?

No. A package that faults is reported and skipped; unrelated packages still load.

Legacy Mods

Can I run my old mods?

Data mods convert. Run them through the converter to produce a *.FUSE package — see `MIGRATION_FROM_LEGACY.md`.

Script mods that ship .dll logic do not convert. Neither do rolling stock, locomotive, and car mods, except audio definitions FUSE can import.

Can I run the legacy and converted versions of a route together?

No — not for normal play. Both claim the same object ids and you get duplicated track, industries, or buildings. `/fuse.conflicts` reports it. Loading both is supported only as a deliberate conflict test.

Do I need to reconvert after updating?

Reconvert when the converter version changes, the schema version changes, a converter repair became an actual runtime fix, or the legacy package itself was updated.

Multiplayer

Does FUSE work in multiplayer?

In compatibility mode only. FUSE does not sync package contents over the network. Every host and client applies its own local package stack — the same expectation legacy Railloader had, where everyone installs the same mods.

What do all players need?

The same FUSE build, the same enabled package list, and the same load order, installed locally on each machine.

What happens if a client's mods don't match?

FUSE does not negotiate the mismatch. A mismatched client can desync visually or operationally. Non-host clients log a warning the first time they apply runtime world changes.

A server that would rather refuse a mismatched client than let it desync can set `BlockNonHostMultiplayerClientWorldApply` to true. It is off by default so private tests behave like RailLoader. See `SETTINGS.md`.

Saves And Safety

Will FUSE break my saves?

Back them up before installing, updating, or changing your package list. A save made with packages loaded depends on the world those packages created — removing a package that a save relies on changes what that save can load.

Can I remove FUSE later?

Yes. Remove your *.FUSE packages first, then FUSE itself, then restore your previous mods and save backup. Order matters — see `GETTING_STARTED.md`.

Are the experimental commands safe?

`/fuse.reapply` and `/fuse.restore` are for testing and recovery, not normal play. Both refuse to run while a map is loaded unless you pass `--force`, because re-applying mid-session can destabilize a running save. Back up first. Restarting the game is the safer option in every case where you are not specifically testing reload behavior.

Performance

FUSE is slow to load / the game stutters.

Turn on `EnableFrameSpikeDiagnostics` to log frames over the threshold, then check `FUSE.log` for what is spiking. It is a measurement tool — it identifies the cause rather than fixing it. See `SETTINGS.md`.

Note that load time scales with how much your packages actually change: track segments, scenery, and industries all cost time to apply.

Should I leave the debug overlays on?

No. The track, scenery, and world-label overlays draw every frame and exist for authoring and debugging. Leave them off during normal play.

Authoring

How do I make a package?

Start with `PACKAGE_AUTHOR_GUIDE.md`, then the schema at `schemas/FUSE_JSON_SCHEMA.md`.

Is there an editor?

The standalone desktop editor ships as its own download — see `EXTERNAL_EDITOR.md`. The full in-game editor workflow is not part of this release.

How do I check my package is valid?

`/fuse.validate <modId>` re-runs the validator against a loaded package and prints errors and warnings with the offending field. The `fuse-convert` CLI also validates as part of conversion and can fail a build on errors, which is the better fit for CI.

Reporting Problems

What should I include in a bug report?

- `FUSE.log` and `Player.log`
- The output of `/fuse.report json`
- `/fuse.loaded` when package state matters
- `/fuse.conflicts` when conflicts are reported
- `conversion-report.json` and `conversion-report.md` for the affected package
- Screenshots and your exact mod and package list

Turning on `VerboseApplyReportDetails` before reproducing usually identifies the offending object without a second attempt.

Where do I report it?

GitHub Issues. Issues reproducible with a minimal converted package, or against a documented legacy route the converter covers, get priority.

Migrating From Legacy Mods

FUSE replaces the legacy Railroader modding stack. This guide covers moving an existing install — Railroader, Strange Customs, ConfusingSupplements, For Your Convenience, or Alina's Map Mod — over to FUSE packages.

If you are setting up a fresh install with no legacy mods, you do not need this page. See `GETTING_STARTED.md`.

What Changes

Legacy data mods are **converted**, not loaded as-is. The converter reads a legacy mod folder and writes a *.FUSE package next to it, containing the same content expressed in the FUSE schema plus a conversion report.

What carries over:

Legacy content	Status under FUSE
Track / route JSON	Converted, one FUSE data file per source JSON
Horn / whistle / bell packs	Converted, audio files copied
Strange Customs asset packs	Converted to a FUSE asset wrapper
World scenery, scene clones, map masks, splines	Converted
Industries, loaders, stations, team/repair tracks	Converted
Progression sections and delivery phases	Converted

What does not carry over:

- **Arbitrary script mods.** A legacy mod that ships .dll logic is not a data package, and the converter cannot translate its behavior. It warns and leaves the binaries alone.
- **Rolling stock, locomotive, and car mods**, except audio definitions.
- **Signals.**
- **Three-way switches** — Railroader does not support them as standard graph switches, so legacy content authoring one has an authoring problem that predates FUSE.

The Critical Rule

Do not run a legacy route and its converted FUSE equivalent at the same time.

Both stacks will try to own the same object ids, and you get duplicate track, duplicate industries, or duplicate buildings. `/fuse.conflicts` reports exactly this situation, and it is the single most common self-inflicted problem when migrating.

The same applies to the loaders themselves. FUSE detects a leftover `Railloader.dll` or `Railloader.Interchange.dll` in the install and warns about it, both on disk and among loaded assemblies. Take the warning seriously — it means two loaders are live at once.

Loading both is supported only as a deliberate conflict test.

Migration Steps

1. Back up first

Back up your saves and your entire Mods folder before touching anything. This is the step people skip and regret.

2. Inventory what you have

List your legacy mods and sort them into three groups:

- **Data mods** (routes, scenery, industries, audio, asset packs) — these convert.
- **Script mods** (.dll logic) — these do not convert; check whether a FUSE-native equivalent exists.
- **Rolling stock** — out of scope except audio.

3. Convert the data mods

Point the converter at each legacy mod folder. The drag-and-drop route:

```
.\FUSE-Converter.exe
```

Or the .NET CLI, which converts and validates in one pass:

```
fuse-convert "C:\Path\To\LegacyMod" --out "C:\Steam\steamapps\common\Railroader\Mods" --format all
```

--batch converts an entire folder of legacy mods at once. Full options are in FUSE_CONVERTER.md.

4. Read the conversion report

Every converted package gets conversion-report.json and conversion-report.md. **Read them.** The converter classifies each source concept rather than silently dropping it:

Outcome	What it means for you
converted	Maps directly to FUSE. Nothing to do.
repaired	Invalid legacy data was fixed. Worth a look, but generally fine.
preserved	Kept in the package but without full runtime behavior yet.
dependency-required	Works once you install the named dependency.
unresolved	A reference could not be resolved — needs manual repair.
unsupported	FUSE intentionally does not support this. Named in the report.
error	Conversion failed. Do not ship or rely on this package.

Warnings do not automatically mean a package is broken. They mean the conversion needs a look before you call it verified.

The lossy areas that most often need manual verification: legacy formulaic formula values, progression interchangeTransfers, unknown spline or object handlers, unknown industry component types, and script binaries.

5. Remove the legacy versions

Once the converted package is in place, remove or disable the legacy original. Both cannot be active.

Remove the legacy loader itself only after every data mod you care about has been converted and verified — you may want to go back.

6. Install dependencies

Converted route packages frequently depend on asset packs. Install them as real asset packs. Do not work around a missing asset by aliasing it to a different name; if the correct pack exists, install it. `/fuse.assets` lists what FUSE actually found.

7. Verify in game

Load a map and check:

```
/fuse.report  
/fuse.loaded  
/fuse.conflicts
```

A clean migration shows zero faults and zero conflicts. Then look at the world — location list, progression visibility, and terrain masks are the things that look wrong in ways a report will not catch.

Per-Mod Notes

Railloader

The legacy loader. FUSE reads Railloader's package metadata directly, including `RailLoadPriority`, `RailLoadAfter`, and `RailLoadBefore` for load ordering, so ordering relationships you already established carry across.

Remove `Railloader.dll` and `Railloader.Interchange.dll` once you have migrated — FUSE warns while they remain.

Strange Customs

Asset packs convert to a FUSE asset wrapper with `FuseAssetPacks` declared in `Info.json`. Mixin files convert through the `mixinto` mechanism; a missing `mixinto` requirement skips only that fragment rather than faulting the whole stack.

ConfusingSupplements / For Your Convenience

Data content converts. Any script-driven behavior does not — check the conversion report for what was classified unsupported.

Alina's Map Mod

Map content converts. As with any route, do not run the legacy version alongside the converted one.

When To Reconvert

Reconvert a package when:

- The FUSE converter version changes.

- The schema version changes.
- A converter repair changed from warning-only to an actual runtime fix.
- The legacy package was updated or redownloaded.

If Something Goes Wrong

Restore your backup — that is what it is for. Then narrow the problem down to a single package by disabling packages until the symptom disappears, and file a report with the conversion report for that one package attached.

TROUBLESHOOTING.md maps symptoms to the diagnostic command that explains them.

Related

- FUSE_CONVERTER.md — full converter reference
- GETTING_STARTED.md — installing FUSE itself
- KNOWN_ISSUES.md — known legacy interaction problems

Settings Reference

FUSE has 28 settings. Most players never need to change any of them — the defaults are the supported configuration, and the majority of these switches are diagnostic tools rather than features.

Where Settings Live

Settings resolve from three layers, each overriding the one before it:

- 1 **Built-in defaults** — compiled into FUSE. Used when nothing else specifies a value.
- 2 **Info.json** — the Settings object in Railroader/Mods/FUSE/Info.json. The shipped file lists 13 of the 28; any setting absent from the file falls back to its built-in default, and you can add a missing one by name to set it.
- 3 **User settings** — settings.json, written by the in-game FUSE menu.

The user settings file wins over Info.json. It lives at:

```
%USERPROFILE%\AppData\LocalLow\Giraffe Lab LLC\Railroader\FUSE\settings.json
```

Changing a setting through the FUSE menu writes it to that file, so an edit made in-game survives a mod update that replaces Info.json. Conversely, editing Info.json has no visible effect if the same key was already set in-game — clear it from settings.json first.

If Info.json is missing or fails to parse, FUSE logs the problem and falls back to defaults for every setting rather than loading a partial configuration.

FUSE writes the fully resolved setting values to FUSE.log at startup, so the log is the authoritative record of what was actually in effect for a given session.

Player Settings

The handful of settings that change how the game plays or looks.

EnableEnhancedLoadingScreen

Default: true — the only setting that ships enabled.

Replaces the stock "Loading..." screen with staged progress and a current-step label that stays up until FUSE's post-load pipeline finishes. Turning it off falls the whole feature back to the untouched stock screen.

DecoupleVisualConditionLimits

Default: false

Controls the per-car Visual Condition slider. When off, the slider can only make a car look *more* worn than its mechanical condition. When on, the visual override applies verbatim, so a mechanically worn car can be made to look fresh.

RandomizeVisualConditionOnSpawn

Default: false

Randomizes each car's visual condition when it spawns, within the range below.

`RandomVisualConditionMin / RandomVisualConditionMax`

Defaults: 0.6 and 1.0, clamped to the range 0–1.

The spawn randomization range, used only when `RandomizeVisualConditionOnSpawn` is on. The default span — mostly presentable cars up to factory-fresh — mirrors the legacy behavior players already expect.

`ShowLegacyModsInUmm`

Default: false

Adds a synthetic UMM row for every legacy data package, putting them in UMM's global mod list. Kept opt-in for a performance reason: UMM walks that list from several per-frame callbacks. FUSE's own Mods page is the primary place to inspect legacy packages.

Multiplayer

`BlockNonHostMultiplayerClientWorldApply`

Default: false

When on, non-host clients are blocked from applying runtime world changes entirely. Off by default so private multiplayer tests behave like RailLoader, where every player is expected to have the same mods installed locally.

FUSE does not sync package contents over the network in either mode. Turning this on is a strict guard for servers that would rather refuse a mismatched client than let it desync. See [FAQ.md](#) for what FUSE does and does not guarantee in multiplayer.

Logging And Reports

`MirrorInfoToPlayerLog`

Default: false · *not in the shipped* `Info.json`

Mirrors FUSE's info-level log lines into Unity's `Player.log` in addition to `FUSE.log`. Useful when correlating FUSE activity against base-game or other-mod log output on one timeline.

`VerboseApplyReportDetails`

Default: false

Expands the per-package detail in the apply report. Turn this on before reproducing an issue for a bug report — the extra detail usually identifies the offending object without a second reproduction.

`MirrorAssetPacksToLocalLow`

Default: false

Mirrors discovered asset packs into the `LocalLow` folder. A compatibility aid for setups where asset packs are not found in their normal location.

Diagnostic Overlays

Author and debugging tools. All default to off, and none of them are intended to be left on during normal play — the overlays draw every frame.

ShowAdvancedHealthDetails

Default: false

Expands the FUSE menu's health display with advanced detail.

ShowTrackDebugOverlay / ShowTrackDebugSpanPaths

Default: false · *neither is in the shipped Info.json*

Draws the track debug overlay, and within it the span paths. ShowTrackDebugSpanPaths is a sub-toggle — it needs the overlay on to show anything.

ShowSceneryDebugOverlay / ShowSceneryDebugAdvanced

Default: false · *neither is in the shipped Info.json*

The scenery equivalent: an overlay plus its advanced-detail sub-toggle.

ShowWorldLabelsOverlay

Default: false · *not in the shipped Info.json*

The master switch for in-world labels on FUSE objects. The five per-kind toggles below only matter when this is on.

World Label Kinds

Setting	Default
WorldLabelsShowScenery	true
WorldLabelsShowSceneClones	true
WorldLabelsShowIndustries	true
WorldLabelsShowTrackNodes	false
WorldLabelsShowTrackSegments	false

None appear in the shipped Info.json.

The per-kind defaults are deliberately asymmetric: the three that authors usually want are on, so flipping the master switch shows something useful immediately, while the two dense kinds stay off. A typical map has over a thousand track nodes, and enabling those labels paves the screen. Opt into them individually when you actually need them.

Performance Diagnostics

EnableFrameSpikeDiagnostics

Default: false

Logs a line whenever a frame takes longer than the threshold below. This is a measurement tool for attributing stutter, not a fix — it does not make anything faster. Takes effect immediately, without a restart.

FrameSpikeThresholdMs

Default: 100, clamped to **20–500**.

The frame duration that counts as a spike. 100 ms is roughly a clearly felt hitch at any refresh rate without flagging ordinary frame-time noise.

The bounds are enforced on every path that writes this value, so no source can set one outside the range: below 20 ms, ordinary frames at low fps would log as spikes; above 500 ms you are measuring stalls, not spikes. A non-numeric or NaN value degrades to the default rather than escaping the clamp.

EnableSceneryCullingDiagnostics

Default: false

Logs scenery culling decisions. Useful when scenery pops in late or fails to appear.

ForceConstrainedVramMode

Default: false

Forces the constrained-VRAM scenery policy on hardware that would not otherwise trigger it — a test override for reproducing low-VRAM behavior on a larger card.

Restart the game before collecting a comparison capture. The setting persists, but the policy it selects is chosen during startup.

EnableNativeLeakStackTraces

Default: false

Enables Unity's native-allocation leak stack traces. These are process-wide and expensive, so keep them off unless actively chasing a native leak. FUSE restores the host's prior mode when it unloads.

Experimental

Off by default and not part of a supported configuration. Enabling one is a testing decision — back up your save first.

EnableExperimentalEarlyScenePathSuppression

Default: false

Suppresses base-game scene paths early in load, with an 8-second timeout. Cannot be re-enabled mid-session once a map is loaded.

EnableTargetedTerrainInvalidation

Default: false · *not in the shipped* Info.json

Narrows the post-apply terrain rebuild to only the tiles FUSE actually touched, instead of a full teardown and reload. A significant load-time win, but timing-sensitive because masks load asynchronously, so it stays off until validated in-game. It falls back to the full rebuild whenever no footprint was captured.

Settings Not In The Shipped Info.json

These 15 resolve to their built-in defaults unless you add them to the Settings object by name or set them through the FUSE menu:

MirrorInfoToPlayerLog, EnableTargetedTerrainInvalidation, ShowTrackDebugOverlay, ShowTrackDebugSpanPaths, ShowSceneryDebugOverlay, ShowSceneryDebugAdvanced, ShowWorldLabelsOverlay, WorldLabelsShowScenery, WorldLabelsShowSceneClones, WorldLabelsShowIndustries, WorldLabelsShowTrackNodes, WorldLabelsShowTrackSegments, RandomizeVisualConditionOnSpawn, RandomVisualConditionMin, RandomVisualConditionMax.

Related

- [CONSOLE_COMMANDS.md](#) — the commands these overlays complement
- [TROUBLESHOOTING.md](#) — which diagnostic to enable for which symptom

Console Command Reference

FUSE registers its commands with Railroader's in-game console. Open the console, type the command, and press Enter. Every command returns text in the console; the `dump*` family also writes a file to the main Railroader folder.

All commands are safe to run while a map is loaded **except** the two experimental recovery commands at the end of this page.

Quick Index

Command	Purpose
<code>/fuse.report</code>	Last map-load report (human or JSON)
<code>/fuse.loaded</code>	Loaded packages and applied/faulted state
<code>/fuse.conflicts</code>	Recorded ownership collisions
<code>/fuse.validate</code>	Re-run the validator for one package
<code>/fuse.graph</code>	Track graph summary
<code>/fuse.groups</code>	Runtime track groups
<code>/fuse.segments.audit</code>	Segment renderer/visibility audit
<code>/fuse.track.rebuild</code>	Force a full track geometry rebuild
<code>/fuse.operations</code>	Loads, industries, loaders, stations, turntables
<code>/fuse.progressions</code>	Progression sections, features, delivery phases
<code>/fuse.assets</code>	Discovered asset pack folders
<code>/fuse.suppressions</code>	Active world suppressions
<code>/fuse.patches</code>	Harmony patches applied or skipped
<code>/fuse.dumpgraph</code>	Write the captured original graph
<code>/fuse.dumpruntimegraph</code>	Write the active post-FUSE graph
<code>/fuse.dumpmandelas</code>	Write scene clone and world path data
<code>/fuse.dumpprogression</code>	Write the live progression graph
<code>/fuse.reapply</code>	Experimental — re-apply resident definitions
<code>/fuse.restore</code>	Experimental — reload from disk and reapply

Status And Packages

`/fuse.report`

Shows the last FUSE map-load report — the same summary the startup toast abbreviates.

```
/fuse.report
```

Pass `json` for machine-readable output, which is the better form to paste into a bug report because it does not wrap or truncate:

```
/fuse.report json
```

```
/fuse.loaded
```

Lists every loaded FUSE package with its applied/faulted state. This is the first command to run when a package's content is missing from the world — a package that faulted during apply still appears here, with the fault recorded.

A faulted package does not stop unrelated packages from loading.

```
/fuse.conflicts
```

Lists registry conflicts — cases where two packages claimed ownership of the same object id. An empty list is the healthy result.

Conflicts are the expected outcome of loading a legacy route and its converted FUSE equivalent at the same time. See `MIGRATION_FROM_LEGACY.md`.

```
/fuse.validate
```

Re-runs the FUSE validator against one already-loaded package and prints errors and warnings with the offending field and error code.

```
/fuse.validate <modId>
```

The `<modId>` argument is required; without it the command prints its usage line. If the id is not loaded, the command says so rather than failing silently — check the spelling against `/fuse.loaded`.

Each line is formatted as `[error]` or `[warn ]` followed by the field, message, and code.

Track Graph

```
/fuse.graph
```

Summarizes the active Railroader graph alongside FUSE's own track definitions. Use it to confirm FUSE applied the node/segment counts you expect.

```
/fuse.groups
```

Lists the runtime track groups discovered on the active graph. Group ids (`s2`, for example) are the filter values

`/fuse.segments.audit` accepts.

```
/fuse.segments.audit
```

Audits FUSE-tracked segments for visual renderer state — the command to reach for when track exists in the graph but nothing is drawn on screen. It reports, per group, how many segments have no renderer, have renderers but none enabled, or are inactive, and flags which segments FUSE claims versus vanilla.

```
/fuse.segments.audit  
/fuse.segments.audit s2  
/fuse.segments.audit s2 verbose
```

The first non-flag argument filters by group id. Adding `verbose` (or `-v` / `--verbose`) emits per-segment lines instead of group summaries — expect long output on a full route.

Requires a populated graph; before a map finishes loading it reports that and stops.

`/fuse.track.rebuild`

Invalidates every segment's cached bezier curve and forces a full `TrackObjectManager` rebuild.

This is the recovery for the "switch tracks do not intersect" geometry failure, where stale cached curves no longer reflect post-migration node positions and rotations. Invalidating the curves forces the next access to recompute against the current node transforms.

The command reports how many curves it invalidated and the segment count before and after. If previously-invisible switches and their connected segments render afterward, stale curve caches were the cause.

Operations, Progression, And World

`/fuse.operations`

Summarizes loads, industries, industry components, loaders, stations, and turntables. Run this for a wrong-looking company window, Locations tab, or industry.

`/fuse.progressions`

Summarizes progression sections, map features, and delivery phases. Run this when an object is visible earlier than its progression should allow.

`/fuse.assets`

Lists the FUSE asset pack folders discovered for direct `PrefabStore` loading. If scenery is missing, confirm here that the required pack was actually found before assuming the package is at fault.

`/fuse.suppressions`

Lists active FUSE world suppressions — base-game world objects a package asked FUSE to hide.

`/fuse.patches`

Lists the Harmony patch classes FUSE applied or skipped, with a failure reason for each skipped patch. Worth including in any bug report that involves another mod, since a skipped patch usually means a conflict with something else in the load order.

Dump Commands

Each writes a JSON file to the main Railroader folder. These files are large; attach them to bug reports rather than pasting their contents.

`/fuse.dumpgraph`

Writes FUSE's captured *original* Railroader track graph to `FUSE-original-graph.json` — the graph as it existed before FUSE applied anything.

`/fuse.dumpruntimegraph`

Writes the active *post-FUSE* track graph to `FUSE-runtime-graph.json`.

Diffing this against `FUSE-original-graph.json` is the standard way to see exactly what a package changed about the track.

`/fuse.dumpmandelas`

Writes loaded scene-clone definitions and the current World scene paths to `FUSE-mandelas.json`. Use it when a scene clone resolves to the wrong building — compare the recorded source path against the base game scene path.

`/fuse.dumpprogression`

Writes the live progression graph to `FUSE-progression.json`, covering features, sections, track-group ownership, areas, industries, and passenger stops.

Experimental Commands

These two exist for testing and recovery. They are not part of normal play, and each logs a warning the first time it is used in a session.

Both refuse to run while a map is loaded unless you pass `--force`. That guard is deliberate: re-applying definitions mid-session can destabilize a running save. Back up the save before overriding it.

`/fuse.reapply` **(experimental)**

Rebuilds FUSE's caches and re-applies the definitions already resident in memory. It does not re-read anything from disk. Reports how many definitions it applied.

```
/fuse.reapply
/fuse.reapply --force
```

`/fuse.restore` **(experimental)**

The heavier operation: unloads every package, clears all caches, reloads packages from disk, rebuilds caches, and re-applies. Use it after editing package files on disk and wanting them picked up without a game restart.

Reports both how many packages loaded from disk and how many applied to the runtime.

```
/fuse.restore
/fuse.restore --force
```

Restarting the game is the safer path in every case where you are not specifically testing reload behavior.

Related

- `TROUBLESHOOTING.md` — which command to run for which symptom
- `SETTINGS.md` — the debug overlays that complement these commands
- `PACKAGE_AUTHOR_GUIDE.md` — using these while authoring

Troubleshooting

First Checks

- 1 Start the game and load the map.
- 2 Read the startup FUSE toast.
- 3 Run `/fuse.report`.
- 4 If the report shows faults, conflicts, asset issues, graph issues, or transfer skips, gather the files listed below.

Files To Attach

Attach these for almost every report:

- `%USERPROFILE%\AppData\LocalLow\Giraffe Lab LLC\Railroader\FUSE.log`
- `%USERPROFILE%\AppData\LocalLow\Giraffe Lab LLC\Railroader\Player.log`
- The affected package folder or zip when possible
- The package conversion report: `conversion-report.json`
- The package conversion report: `conversion-report.md`
- Screenshots of visible map, scenery, company window, or progression issues

Also include console output or screenshots for:

- `/fuse.report`
- `/fuse.loaded`
- `/fuse.conflicts`
- `/fuse.graph` for graph or track issues
- `/fuse.operations` for industries, stations, loaders, or turntables
- `/fuse.progressions` for map feature/progression visibility issues
- `/fuse.assets` for missing scenery assets

Common Symptoms

Faulted Package

Run `/fuse.loaded` and `/fuse.report`. Check `FUSE.log` for the package id and phase. Package failures should not prevent unrelated packages from loading.

Unknown Scenery Asset

Run `/fuse.assets` and verify the required asset pack is installed. Do not replace the missing asset with a guessed alias unless the legacy source actually used the alias.

Missing Or Wrong Building

Check whether it is a regular asset pack item or a scene clone. For scene clones, run `/fuse.dumpmandelas` and compare the source path with the base game scene path.

Bad Track, Missing Span, Or Broken Segment

Run `/fuse.dumpgraph` and `/fuse.dumpruntimegraph`. The files are written to the main Railroader folder as `FUSE-original-graph.json` and `FUSE-runtime-graph.json`.

Company Window Or Location List Looks Wrong

Run `/fuse.operations`. Include screenshots of the Locations tab and the route map. Area ordering may depend on converted area order data and runtime base-game ordering.

Progression Object Visible Too Early

Run `/fuse.progressions`. Include the affected package, progression id, map feature id, and screenshot of the object.

Audio Missing Or Wrong

Check that the converted audio package is installed and that the source audio files were copied beside the generated `audio.fuse.json`.

When To Reconvert

Reconvert packages when:

- The FUSE converter version changes.
- The schema version changes.
- A converter repair changed from warning-only to an actual runtime fix.
- A legacy package was updated or redownloaded.

What Not To Do

- Do not load the legacy and FUSE versions of the same route for normal play.
- Do not delete warnings from converter reports just to get a clean report.
- Do not patch a missing asset to a different name if the correct asset pack exists.
- Do not treat unsupported three-way switches as a FUSE runtime bug.

Known Issues

Current limitations, unsupported content, and known interactions. For debugging a specific symptom, see [TROUBLESHOOTING.md](#).

Out Of Scope

Not defects — content FUSE does not aim to support in this release.

- **The full public in-game editor workflow.** The standalone external editor ships instead — see [EXTERNAL_EDITOR.md](#).
- **Rolling stock, locomotive, and car mods**, except audio-only horn, whistle, and bell packs, which convert.
- **Arbitrary legacy script mods.** Only data, asset, audio, and supported runtime component packages convert.
- **Signals.**
- **Mid-session scene-path suppression re-enable.** Once a map is loaded, the setting cannot be turned back on for that session.

Game Limitations

Constraints from Railroader itself, not from FUSE.

- **Three-way switches are not supported.** Railroader does not treat them as normal graph switches. Legacy content authoring a three-way switch shape has an authoring problem that predates FUSE, and should be reported as such rather than as a FUSE graph bug.
- **Some legacy packages ship broken passenger stop or depot definitions** that were already broken under the original legacy stack. FUSE reports these clearly and avoids making them worse, but it cannot invent the missing data.

Experimental Features

Off by default. Enabling one is a testing decision — back up your save first. See [SETTINGS.md](#).

- **Early scene-path suppression** (`EnableExperimentalEarlyScenePathSuppression`).
- **Targeted terrain invalidation** (`EnableTargetedTerrainInvalidation`) — a significant load-time win, but timing-sensitive because masks load asynchronously. Falls back to a full rebuild when no footprint was captured.
- **Runtime authoring mutations.**
- `/fuse.reapply` **and** `/fuse.restore` — recovery and testing commands that refuse to run mid-session without `--force`.

External Mod Conflicts

- **Do not load a legacy route and its converted FUSE route at the same time** unless you are deliberately testing conflicts. Both claim the same object ids. `/fuse.conflicts` reports the collision.
- **Legacy loaders, AMM, Strange Customs, and RailLoader can create duplicate objects** when used alongside converted packages for the same route. FUSE warns when it finds a leftover `Railloader.dll` or `Railloader.Interchange.dll`.

- **Custom industry components load only when the owning assembly is installed.** A package referencing a component type from an assembly you do not have reports the missing dependency rather than silently dropping the component.

See `MIGRATION_FROM_LEGACY.md` for the migration order that avoids most of this.

Multiplayer

FUSE uses legacy-style multiplayer compatibility mode.

- **Every player needs the same FUSE build, enabled package list, and load order**, installed locally. FUSE does not sync package contents over the network.
- **Mismatches are not negotiated.** A mismatched client can desync visually or operationally. Non-host clients log a warning on their first runtime world apply.
- **Strict client blocking is available** through `BlockNonHostMultiplayerClientWorldApply`, disabled by default so private tests behave like `RailLoader`.

Areas Needing Visual Verification

These apply correctly by every automated check available, but their *appearance* depends on interactions that only a human looking at the map can confirm. If something looks wrong here, it is worth reporting even when the load report is clean:

- Location and area ordering in the company window, which depends on both converted area order data and base-game runtime ordering
- Progression visibility timing
- Map mask visuals — terrain flattening, tree cutting, and height masks

Reporting

Include `FUSE.log`, `Player.log`, `/fuse.report.json`, and the conversion report for the affected package. Full checklist in `TROUBLESHOOTING.md`.

Installer

`fuse_installer.py` installs mod packages from zip files into the base folder's Mods directory. It is designed for a bundled `FUSE-Installer.exe`, so users can drop zip files beside the executable and run it from the base folder.

Download

The published `FUSE-Installer.exe` is distributed via GitHub Releases: <https://github.com/F-U-S-E-E/FuseDevelopmentGroup/releases>. The `dist/` folder is gitignored, so building locally produces a private copy at `dist\FUSE-Installer.exe`.

The installer inspects zip structure and manifest JSON only. It does not import, execute, or depend on any package code.

Supported zips

- UMM packages with `Info.json`.
- FUSE data packages with `Info.json` and FUSE data markers such as `FuseDataFiles`, `FuseAssetPacks`, or a FUSE requirement.
- Supported legacy data packages with `Definition.json` plus data JSON containing world, track, operations, or progression entries.
- Multi-package zips laid out as `Mods/PackageA/...` and `Mods/PackageB/...`

User flow

- 1 Put `FUSE-Installer.exe` in the base folder.
- 2 Drop one or more `.zip` files in that same folder.
- 3 Run `FUSE-Installer.exe`.
- 4 The installer writes packages to `Mods\<package id>`.

Existing folders are skipped by default. Run with `--replace` to move an existing folder into `Mods\ModBackups\FUSEInstaller\<timestamp>` before installing the new copy.

Command examples

Install every zip in the current folder:

```
.\FUSE-Installer.exe
```

Install explicit zips:

```
.\FUSE-Installer.exe .\MyPackage.zip .\OtherPackage.zip
```

Inspect without writing:

```
.\FUSE-Installer.exe --dry-run
```

Install from a different location:

```
.\FUSE-Installer.exe --game-dir "D:\Games\BaseFolder" --inbox "D:\Downloads\Mods"
```

Archive processed zips after successful installs:

```
.\FUSE-Installer.exe --archive-zips
```

Building the exe

From the repo root:

```
powershell -ExecutionPolicy Bypass -File .\tools\build_installer_exe.ps1
```

If PyInstaller is not installed for the active Python environment:

```
powershell -ExecutionPolicy Bypass -File .\tools\build_installer_exe.ps1 -InstallPyInstaller
```

The output is `dist\FUSE-Installer.exe`.